DSA REFERENCE

# Fast and Slow Pointers Problem Ladder

A sequenced practice ladder from warm-up to interview stretch.

Move two traversals at different speeds so their relative motion reveals cycles, midpoints, or phase boundaries.

VERSION 1.0 REVIEW

REVIEWED THROUGH 2026-07-18

STATUS HUMAN REVIEW REQUIRED

DSA / FAST-SLOW-POINTERS

DSA

# Fast and Slow Pointers Problem Ladder

**CORE IDEA**

**Move two traversals at different speeds so their relative motion reveals cycles, midpoints, or phase boundaries.**

### Recognition cues

+ linked-list cycle
+ middle node
+ repeated state transition
+ constant-space sequence detection

### Implementation sequence

+ guard every multi-step dereference
+ explain relative speed
+ separate detection from entry location
+ test odd and even lengths

## Invariant

Both pointers follow the same deterministic next relation; only their speed or restart position differs.

## Worked reasoning

After slow and fast meet in a cycle, reset one pointer to head and move both one step to find the entry.

## Complexity

O(n) time and O(1) space for linked structures or deterministic state machines.

## Failure modes

- [ ] unsafe fast.next.next access
- [ ] returning the meeting point as the entry
- [ ] misstating even-length midpoint
- [ ] using it when transitions are nondeterministic

Practice ladder

# Fast and Slow Pointers Problem Ladder: referenced problems

Open the official problem page, classify the pattern, write the invariant, and predict the complexity before coding.

| # | Problem | Deliberate-practice prompt |
|---|---|---|
| 1 | **Linked List Cycle** | State the cue, invariant, edge case, and proof before implementation. |
| 2 | **Linked List Cycle II** | State the cue, invariant, edge case, and proof before implementation. |
| 3 | **Middle of the Linked List** | State the cue, invariant, edge case, and proof before implementation. |
| 4 | **Happy Number** | State the cue, invariant, edge case, and proof before implementation. |
| 5 | **Find the Duplicate Number** | State the cue, invariant, edge case, and proof before implementation. |

### Before coding

+ Pattern cue
+ State
+ Invariant
+ Boundary convention

### Before finishing

+ Dry run
+ Edge case
+ Time proof
+ Space proof

PUBLISHING STATUS

REVIEW GATE: STABLE TECHNICAL. AI-produced draft. A named human must verify technical accuracy, sourcing, freshness, and editorial quality before publication. Reviewed-through date: 2026-07-18.